Claude Certified Associate Foundations

Exam Preparation Tutorial

A 101-level guide to the domains, objectives, and judgment the CCAO-F exam tests — written for
business professionals, project managers, and AI-curious knowledge workers

The prequel to the Claude Certified Architect – Foundations tutorial

Contents

1. How to Use This Tutorial

This tutorial translates the official Claude Certified Associate – Foundations (CCAO-F) exam guide into plain language. It is the prequel to the Architect – Foundations tutorial in this series: before you learn to build systems with Claude, you learn to work with Claude — prompting it well, checking its output, picking the right product features and models, and staying inside your organization's guardrails. No coding, no APIs, no agent architectures. If you can write a clear email and read a simple table, you have everything you need.

The exam is not a trivia test. Most questions describe a realistic workplace moment — a summary bound for the compliance team, a spreadsheet full of customer data, a prompt that keeps producing mush — and ask: *which of these four actions is the right one?* The wrong answers usually sound plausible: trusting Claude because it sounded confident, throwing the biggest model at everything, uploading the data anyway, or abandoning the task entirely. The right answer is almost always the proportionate, verifiable, policy-compliant move.

Three recurring themes run through all seven domains. If you remember nothing else, remember these:

- **1. Verification is your job.** Claude is probabilistic: its answers are very likely correct, not guaranteed correct — and it sounds equally confident either way. The single biggest domain (Output Evaluation and Validation, 21%) exists to test whether you check before you ship. Self-reported confidence is never an accuracy signal.
- **2. Match the tool to the task.** Fast, low-cost models for high-volume simple work; the most capable models for complex reasoning; Projects for recurring context; research mode for deep investigation; artifacts for reusable deliverables. "Use the most powerful option for everything" is a distractor, every time.
- **3. Enable within policy — don't bypass it, don't abandon the work.** The governance answer that wins removes the risk and keeps the task moving: anonymize the identifiers, use the approved workspace, add a human checkpoint. Uploading regulated data as-is loses; refusing to do the analysis at all also loses.

A fourth idea frames the whole credential: **know when to hand it off.** Associates apply Claude to business and productivity work — and recognize when a need crosses into engineering territory (APIs, agents, custom integrations) that belongs to Claude Architects and Developers. Several questions reward exactly that judgment.

How to study: read this tutorial once end-to-end, then re-read each domain chapter alongside hands-on practice in Claude — the exam assumes regular, real-world use, and the fastest way to internalize each objective is to do it. Finish with the sample-question walkthroughs (Chapter 12) and the official sample items in the exam guide.

2. The Exam at a Glance

The certification validates that you can apply Claude to complete business and productivity tasks with minimal guidance: using built-in platform features to streamline workflows, spotting opportunities to improve processes, balancing quality, efficiency, and cost, and recognizing limitations — including when to escalate to the technical tracks. It is aimed at professionals in operations, marketing, project management, education, communications, and general knowledge work.

2.1 Format and Scoring

Item	Details
Exam code	CCAO-F (Claude Certified Associate – Foundations)
Number of items	60
Item format	**Multiple-choice AND multiple-response** — each multiple-response item states how many responses to select. Read that line before the options.
Time limit	120 minutes — two minutes per item on average
Delivery	Proctored: online proctored or Pearson VUE test center
Scoring	Criterion-referenced, scaled 100–1,000; minimum passing score is 720. You pass against a fixed standard, not against other candidates.
Result	Pass/fail with scaled score, plus percent-correct by domain (the domain percentages are diagnostic only — they do not decide the result)
Exam fee	$99 USD, per attempt
Validity	12 months from award
Prerequisites	None — no required courses, no software-development or API experience needed

2.2 The Seven Domains and Their Weight

Seven content domains are tested. Weighting tells you where to invest study time — Output Evaluation and Validation alone is over a fifth of the exam, and it colors questions in every other domain.

Domain	Weight	One-line summary
1. Prompting and Task Execution	14%	Write, structure, decompose, and iterate prompts; adapt strategy to the task type
2. Output Evaluation and Validation	21%	Spot hallucinations and bias, fact-check, know when humans must review, adapt output for the audience

Domain	Weight	One-line summary
3. Product and Model Selection	12%	Pick the right feature (Projects, research, chat, artifacts) and model (Haiku, Sonnet, Opus); manage context limits
4. Workflow Integration and Solution Design	16%	Find good use cases, redesign or augment processes, iterate solutions, communicate value and limits
5. Configuration and Knowledge Management	12%	Configure Projects with instructions, knowledge, and connectors; keep them current
6. Governance, Risk, and Responsible Use	15%	Data sensitivity, privacy, organizational policy, ethics, appropriate vs. inappropriate use
7. Troubleshooting and Optimization	10%	Diagnose weak prompts and poor outputs; adjust based on feedback; optimize for efficiency

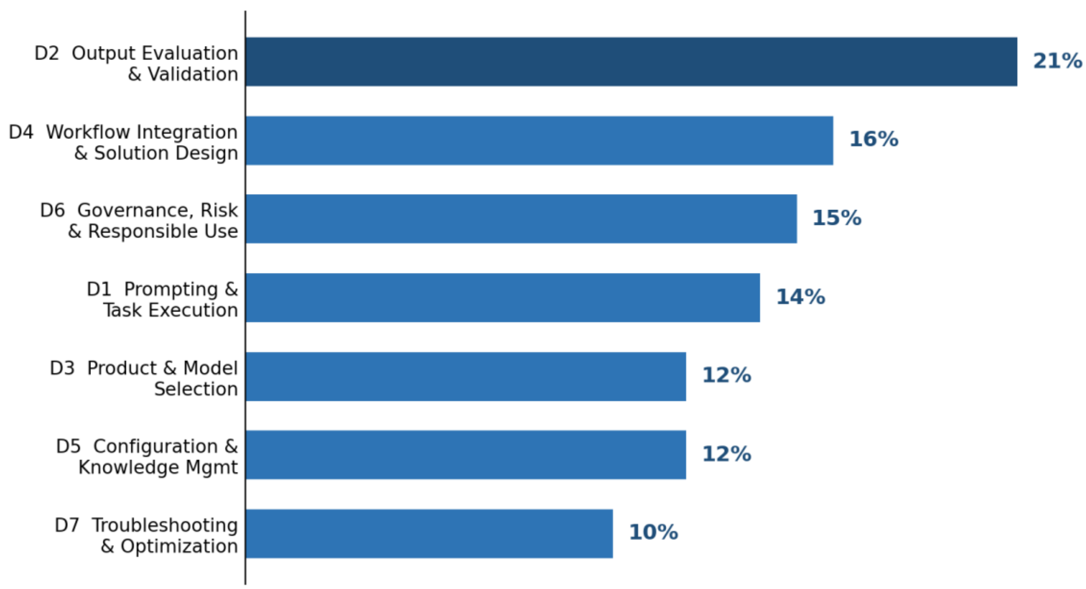


Figure 1 — Where the exam points sit: evaluation and validation is the single biggest domain.

> **Exam tip:** Domains 2 + 6 together are 36% of the exam and share one philosophy: Claude’s speed is only valuable if the output is **trustworthy and permitted**. Judgment about checking and judgment about data are the two muscles this exam works hardest.

2.3 Who This Exam Is For: the Minimally Qualified Candidate

The exam targets a professional who uses Claude as a core productivity tool: someone with regular hands-on experience, foundational applied knowledge of prompt structuring and task orchestration, and familiarity with features like Projects and Artifacts. The profile sits between a casual prompt user and a technical AI practitioner — distinguished by the ability to translate business objectives into effective AI interactions, critically evaluate what comes back, adapt it for different audiences, and recognize when human expertise or escalation is required.

Explicitly out of scope: building against APIs, designing agentic systems, machine learning, and enterprise AI architecture. That work belongs to the Claude Architect and Claude Developer credentials — the “sequels” to this one.

2.4 Where This Exam Sits in the Series

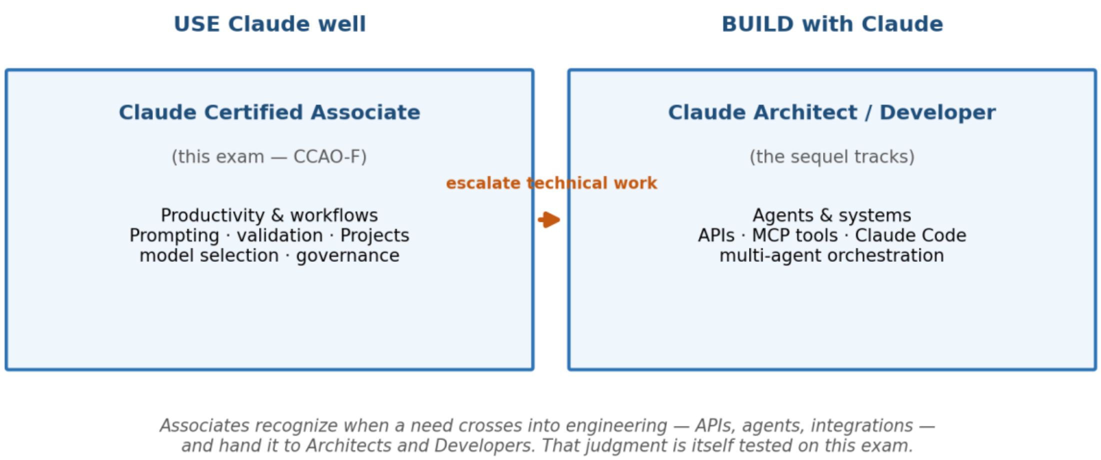


Figure 2 — The Associate track certifies using Claude well; the Architect/Developer tracks certify building with it.

If you later move on to the *Architect – Foundations* tutorial in this series, you will find the same recurring instincts scaled up: “verify before you ship” becomes validation loops and human review workflows; “put standing context in Project instructions” becomes CLAUDE.md files; “decompose the task” becomes multi-agent orchestration. Everything you internalize here transfers.

2.5 Logistics, Retakes, and Renewal

- **Scheduling:** register via the Anthropic Partner Academy, then schedule through Pearson VUE (online proctored or test center). You can cancel or reschedule up to 24 hours before the appointment; changes inside 24 hours — and no-shows or late arrivals — forfeit the fee.
- **Identification:** a valid, unexpired government-issued photo ID whose name exactly matches your registration. Name corrections go to certifications-support@anthropic.com before scheduling.
- **Accommodations:** available for documented disabilities or needs — but they must be requested and approved by Pearson VUE BEFORE you schedule. Do not book the appointment first and ask later.
- **Exam-day rules:** stay in view of the proctor and webcam for the whole session (online); keep the workspace clear of notes, phones, secondary monitors, and study materials; no

communication with anyone; no capturing or reproducing exam content. Phones, smart watches, headphones, and recording devices are prohibited.

- **Retakes:** waiting periods grow with each failed attempt — 14 days after the first, 30 after the second, 90 after the third — with a maximum of four attempts in any rolling twelve months. The fee applies to each attempt. Limits are per exam, so failing this one doesn't block you from registering for a different credential.
- **Renewal:** the credential lasts 12 months. On-time renewal is a free, non-proctored assessment on the Partner Academy; if the credential lapses, you retake the full exam at full fee. One caveat: if exam content changes significantly, Anthropic may require a full retake instead of the renewal assessment.
- **Conduct and appeals:** the exam is delivered under a non-disclosure agreement — content may not be captured, copied, or shared, and misconduct can invalidate results and revoke the credential. Appeals must be filed within 14 days (of the decision, or of the exam date for result concerns) via Pearson VUE support; the standard-setting outcome and individual item content are not appealable.

3. Core Concepts Primer (Read This Before the Domains)

This chapter has two parts. Section 3.1 builds the absolute basics from zero — start here if terms like “context window” or “hallucination” are fuzzy. Section 3.2 is the glossary you will refer back to throughout the domain chapters.

3.1 The Absolute Basics (Start Here if You’re New)

What is Claude? What is a large language model (LLM)?

Claude is an AI assistant built by Anthropic, powered by a large language model — software that reads text and writes text in response. You give it a message (a prompt) and it produces an answer based on patterns learned from enormous amounts of text. Two practical consequences follow, and both are tested constantly: Claude’s answers are very likely correct rather than guaranteed correct, and Claude only knows what is in front of it — your prompt, your files, your Project — plus what it learned in training.

Prompt, instructions, and “system-level” guidance

The prompt is whatever you type in a chat. Instructions are standing guidance you set once — for example, a Project’s custom instructions (“You support the marketing team; write in our house style; UK spelling”) — which silently shape every answer inside that Project. When the exam says “create effective system-level instructions,” it means: write that standing guidance well, so you stop re-explaining the same context in every chat.

Tokens and the context window

Text is measured in tokens (roughly three-quarters of a word). The context window is Claude’s working memory for a conversation — everything it can currently “see”: your messages, its replies, and any files in play. It is finite. Very long chats and very large documents fill it up; when it gets crowded, early details fade and answers drift toward the generic. Recognizing that squeeze — and knowing whether to restart, summarize, or persist context into a Project — is a scored objective in Domain 3.

Hallucination — the exam’s favorite word

A hallucination is confident, plausible, fabricated output: a citation to a subsection that doesn’t exist, a statistic with no source, a person or product detail that was never in the material. Hallucinations are not lies or glitches — they are the natural failure mode of a system that produces likely text. The tell is misplaced specificity: precise-looking numbers, section references, names, and dates that nothing in your source material supports. The cure is never “ask Claude how confident it is”; the cure is verification against an authoritative source.

Probabilistic vs. deterministic — a preview of the whole series

Deterministic means the same input gives the same outcome every time — a calculator, a spreadsheet formula. Probabilistic means usually right, occasionally not — like a skilled colleague working from written guidance. Claude is probabilistic. At the Associate level, the consequence is behavioral: verify outputs, keep humans in the loop for high-stakes work, and never treat fluency as accuracy. (In the Architect track, the same distinction becomes an engineering rule: enforce must-never-fail rules in code, not prompts. Same physics, bigger machinery.)

The Claude product map

The exam expects you to pick the right feature for the moment, so know what each one is for:

Feature	What it is	Reach for it when…
Chat	A single conversation with Claude	Quick questions, one-off tasks, exploration
Project	A workspace bundling custom instructions + uploaded knowledge + related chats that all share them	Recurring work with the same context: a client account, a product launch, a team function
Artifacts	Standalone outputs (documents, tables, code, mini-apps) created beside the chat, easy to iterate on and reuse	Deliverables you will edit, version, and take elsewhere — not throwaway answers
Research mode	A deeper investigation mode where Claude searches, reads, and synthesizes many sources into a cited report	Multi-source questions where coverage and citations matter more than speed
Connectors	Live links to tools such as Google Drive or Gmail, so Claude reads current content at its source	Content that changes often — no stale re-uploads
Skills	Reusable packaged playbooks — instructions Claude loads for a specific, recurring task type (a report format, a review checklist)	The same specialized task recurs and should be done the same way every time
Code execution & file creation	Claude writes and runs code behind the scenes to analyze data and produce real files — spreadsheets, documents, charts	Data analysis beyond mental arithmetic, or deliverables you need as downloadable files
Memory / history	Claude's recall of context across your past conversations	Continuity across sessions — with awareness of what it does and doesn't retain

The Domain 3 blueprint names Projects, research mode, chat, and artifacts explicitly, but the exam guide's own preparation section also points candidates to the documentation for **Memory, Skills, and Code Execution** — so know what each of these is for, even if the deepest questions concentrate on the first four.

The model family: Haiku, Sonnet, Opus

Claude comes in model tiers that trade capability against speed and cost. Haiku is the fastest and cheapest — built for high-volume, straightforward tasks. Sonnet is the balanced everyday workhorse. Opus is the most capable and most expensive — reserved for complex reasoning where quality dominates. Domain 3 tests whether you match the tier to the task instead of defaulting to the top.

3.2 The Exam Glossary

Skim now; refer back as needed.

Term	Plain-language meaning
LLM	Large language model — software that reads and writes text based on learned patterns. Claude is one.
Prompt	The message you send Claude: task, context, source material, format, constraints.
Task decomposition	Breaking one big request into smaller, sequenced steps — each easier to specify, check, and correct.
Iteration	Improving output over rounds by giving specific feedback on what was wrong and what “good” looks like.
Hallucination	Confident, plausible, fabricated content — fake citations, invented numbers, made-up details.
Bias	Systematic skew in output — assumptions about people, one-sided framing, unrepresentative examples.
Context window	Claude’s finite working memory for a conversation; long chats and big files fill it and degrade recall.
Token	The unit text is measured in (~3/4 of a word); context limits are counted in tokens.
Project	A workspace of custom instructions + knowledge files + connectors that every chat inside it inherits.
Project instructions	Standing, system-level guidance written once per Project — role, tone, audience, standards.
Knowledge (in a Project)	Reference documents uploaded to the Project — a snapshot that must be maintained as sources change.
Connector	A live integration (e.g., Google Drive, Gmail) letting Claude read current content at the source.
Artifact	A standalone, reusable output created beside the chat — document, table, code, or small app.
Research mode	A mode for deep, multi-source investigation producing synthesized, cited findings.
Skill	A reusable packaged playbook Claude loads for a specific recurring task type, so it is done consistently.
Code execution / file creation	Claude writes and runs code to analyze data and produce real files (spreadsheets, documents, charts).
Haiku / Sonnet / Opus	Model tiers: fastest-cheapest / balanced default / most capable-most costly.
Human-in-the-loop	A workflow with a mandatory human review step before output takes effect.

Term	Plain-language meaning
Anonymization / redaction	Removing or masking personal or regulated identifiers before sharing data with any tool.
Escalation	Handing work upward — to a human expert for judgment, or to Architects/Developers for technical builds.
Criterion-referenced	Scored against a fixed standard, not a curve; everyone who meets the standard passes.

4. Domain 1: Prompting and Task Execution (14%)

Everything downstream — evaluation, workflows, troubleshooting — is easier when the prompt is right. This domain tests whether you can specify work the way a good manager briefs a capable new colleague: enough context to act, clear success criteria, and a defined deliverable.

Objective 1.1 — Create Effective Prompts for Business and Technical Tasks

What it means: an effective prompt supplies five things — role and context, the task itself, the source material, the output format, and the constraints/audience. Every block you omit is a decision Claude must guess, and guesses are where generic output comes from.

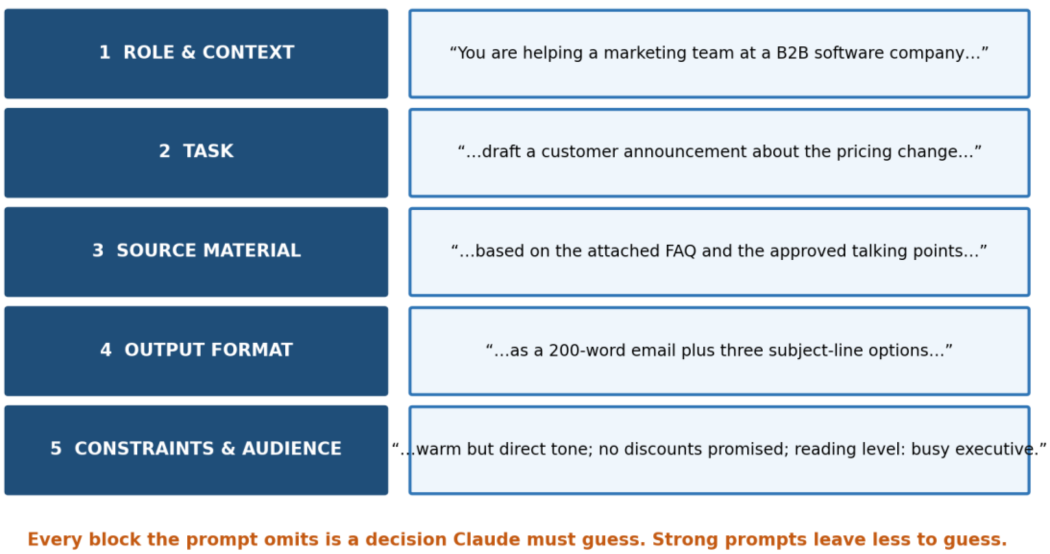


Figure 3 — The five building blocks of an effective prompt. Strong prompts leave less to guess.

See the difference for yourself:

Quality	Prompt
Weak (produces mush)	“Write something about our pricing change.”
Strong	“You’re helping a B2B software company announce a 10% price increase effective 1 September. Using the attached FAQ and approved talking points, draft a 200-word customer email plus three subject-line options. Tone: warm but direct; do not promise discounts or exceptions; audience: busy operations managers who skim.”

The strong version needs no follow-up interrogation: it names the audience, anchors the facts to source material (which also suppresses hallucination), fixes the length and format, and states the boundaries. Notice it is not longer for the sake of length — every sentence removes a guess.

> **Analogy:** Briefing a talented freelancer on day one. “Write something about pricing” gets you a plausible essay you can’t use. A one-paragraph brief with the audience, the source folder, the word count, and the two things they must not say gets you a usable first draft.

Objective 1.2 — Apply Task Decomposition to Structure Complex Requests

What it means: big, multi-part requests fail as one mega-prompt because quality spreads unevenly across the parts and errors are hard to localize. Decomposition breaks the work into focused, sequenced steps — each with its own inputs and its own checkpoint.

Worked example — “prepare our Q3 business review”:

- **Step 1 — Extract:** “From the attached sales export, pull revenue, pipeline, and churn by region into a table.” (Verify the numbers here, while it’s cheap.)
- **Step 2 — Analyze:** “Using the verified table, identify the three biggest changes vs. Q2 and likely drivers.”
- **Step 3 — Draft:** “Turn the analysis into a one-page executive summary in our standard format.”
- **Step 4 — Adapt:** “Now produce a 5-bullet version for the all-hands slide.”

Each step is easy to specify, easy to check, and easy to redo without redoing the rest. Decomposition is also the honest answer to “Claude got the complicated request half right” — the fix is structure, not a bigger model.

> **Exam tip:** When a scenario describes one sprawling prompt producing uneven results, the credited answer is almost always **decompose and checkpoint** — not “add more detail to the mega-prompt,” not “switch models,” and not “give up and do it manually.”

Objective 1.3 — Iterate Prompts to Improve Output Quality

What it means: first drafts are starting points. Skilled iteration has three habits:

- **Give specific feedback, not verdicts.** “Make it better” recycles the same guesses. “The second paragraph buries the deadline — lead with it, and cut the jargon in the intro” steers.
- **Show, don’t describe.** When prose feedback keeps missing, paste one example of the output you want (a past email, a table in the right shape). One demonstrated example beats three paragraphs of description.
- **Change one variable at a time.** If you rewrite the whole prompt each round, you can’t tell which change helped. Adjust the format spec, or the audience line, or the source material — then re-run.

Objective 1.4 — Adapt Prompting Strategy to the Task Type

The exam guide names four task types — analysis, research, drafting, brainstorming — because each rewards a different prompting posture:

Task type	What to supply	What "good" looks like
Analysis	The data itself + explicit evaluation criteria + the decision the analysis feeds	Findings tied to the supplied data, with the reasoning visible enough to audit
Research	Scope, must-cover subtopics, source expectations; use research mode for depth	Multi-source synthesis with citations you can verify
Drafting	Audience, tone, length, format, and an example of house style	A draft that sounds like your organization, not like a template
Brainstorming	The problem + constraints + "quantity first" framing; defer judgment	Many divergent options to prune — not one polished but narrow answer

> **Exam tip:** Task-type mismatch is a common distractor pattern: demanding citations from a brainstorm, or asking for "creative variety" in a compliance analysis. Match the posture to the job.

5. Domain 2: Output Evaluation and Validation (21%)

The biggest domain, and the heart of the credential. Claude produces fluent, confident text at speed; the Associate's value is knowing which parts to trust, how to check the rest, and when the stakes demand a human. Internalize one sentence and a fifth of the exam gets easier: fluency is not accuracy, and confidence is not evidence.

Objective 2.1 — Evaluate Outputs for Accuracy and Completeness

What it means: before any deeper checking, read the output against two questions. Accuracy: is every factual claim supported by the source material or by something you can verify? Completeness: did it actually answer all parts of the request — every question asked, every section required, every case covered? Incompleteness is sneaky because what's present looks great; the failure is what's absent. Re-read your own prompt as a checklist against the answer.

Objective 2.2 — Identify Hallucinations, Inconsistencies, and Biases

The three failure species, and their tells:

Failure	What it looks like	The tell
Hallucination	Confident fabricated specifics: a regulation subsection that doesn't exist, a statistic with no source, an invented customer quote	Misplaced precision — details more specific than anything in your source material
Inconsistency	The output disagrees with itself: a total that doesn't match its line items, "March 3" in one paragraph and "March 30" in another, a recommendation contradicting the stated finding	Cross-check numbers, dates, and names against each other inside the document
Bias	Systematic skew: examples that assume one demographic, one-sided framing of a contested issue, survey synthesis that overweights the loudest respondents	Ask who or what is missing; check whether counter-evidence was represented

Watch a hallucination in the wild (this is the exam guide's own Sample 1 scenario): asked to summarize a new regulation, Claude produces a confident summary citing "subsection 4(b)(ii)." The citation looks rigorous — that is exactly what makes it dangerous. Before that summary reaches the compliance team, the cited subsection must be checked against the official regulation text. Not reworded. Not confidence-rated. Checked.

Objective 2.3 — Apply Fact-Checking and Validation Techniques

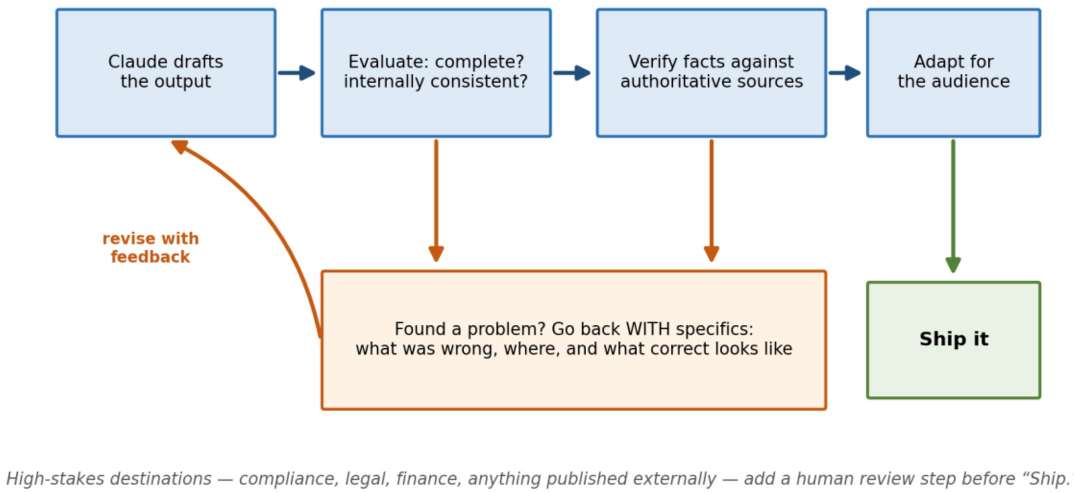


Figure 4 — The verify-before-you-ship loop. Problems travel back with specifics attached.

- **Verify against authoritative sources:** the regulation text itself, the signed contract, the system of record — not a second AI summary of the same thing.
- **Ground the work in supplied documents:** uploading the source and asking Claude to answer from it (and cite where in it) shrinks the space for fabrication dramatically.
- **Spot-check the load-bearing facts:** you rarely need to verify every sentence — verify the numbers, names, dates, and claims the decision actually rests on.
- **Ask Claude to flag its own uncertainty:** "list any claims above you couldn't ground in the attached documents" is a useful triage step — useful for finding what to check, never a substitute for checking.
- **Compare independent drafts:** for important analyses, run the request fresh in a separate chat and compare conclusions. Divergence marks exactly where scrutiny belongs. (In the Architect track this instinct becomes formal multi-instance review — same idea, more machinery.)

Objective 2.4 — Determine When Human Review Is Required

The stakes ladder: match verification effort to the cost of being wrong.

Stakes	Examples	Required diligence
Low	Internal brainstorm, meeting-notes cleanup, first draft you will rewrite anyway	Quick read-through
Medium	Internal report, team documentation, analysis feeding a routine decision	Verify key facts; check completeness against the request
High	Anything for compliance, legal, or finance; external publication; decisions affecting people (hiring, performance, customers)	Full verification against authoritative sources PLUS

Stakes	Examples	Required diligence
		qualified human review before it takes effect

> **Exam tip:** Any option resembling "send it — Claude expressed high confidence" or "ask Claude to rate its confidence and send if high" is a distractor. Self-reported confidence is not calibrated to accuracy; the credited action verifies against an authoritative source or routes to a human. (The same fact reappears in the Architect exam as "never use model confidence as an escalation trigger.")

Objective 2.5 — Edit, Adapt, Refine, and Compare Outputs for the Audience

Validation isn't only about truth — it's about fitness. A technically accurate answer can still fail its audience: too long for an executive, too thin for an engineer, tone-deaf for an upset customer. The tested skills: rewrite for a specified reader ("same facts, 100 words, no jargon, for the CFO"), maintain the organization's voice, and compare candidate versions against explicit criteria rather than vibes. When choosing between two drafts, name the criteria first — accuracy, clarity for this reader, tone, length — then judge both against them.

Objective 2.6 — Organize Information and Select the Right Output Format

Format	Choose it when	Example
Inline chat answer	The value is conversational and immediate; nobody will reuse it	"What does this clause mean?"
Artifact	It's a deliverable — it will be edited, versioned, shared, or reused outside the chat	A proposal draft, a project plan, a comparison table you'll keep updating
Structured data (table / CSV / JSON)	The output feeds another tool or process and consistency of fields matters	Extracted survey themes going into a tracker; contact fields going into a CRM

Curation is part of the objective: the raw output of a research or extraction task is rarely the final shape. Organizing it — deduplicating, grouping, ranking by relevance, choosing table vs. prose per content type — is Associate work, and the exam rewards choosing the format the downstream consumer needs.

6. Domain 3: Product and Model Selection (12%)

A small domain with an outsized theme: fit. The exam repeatedly offers a maximal option — the biggest model, the deepest mode, a full restart — and rewards the candidate who picks the proportionate one.

Objective 3.1 — Select the Right Claude Product Feature

You met the product map in Chapter 3. The tested judgment is recognizing the moment each feature is for:

- **A one-off question** → plain chat. Creating a Project for a single task is overhead without benefit.
- **The third chat this week re-explaining the same client context** → a Project. Move the context into instructions and knowledge once; every future chat inherits it.
- **"What does the market landscape for X look like?" with real depth required** → research mode. A single-pass chat answer will be shallow and thinly sourced.
- **A document you'll revise four times and send onward** → an artifact, so iteration targets the deliverable itself instead of scrolling through chat history.

Objective 3.2 — Differentiate the Claude Model Types

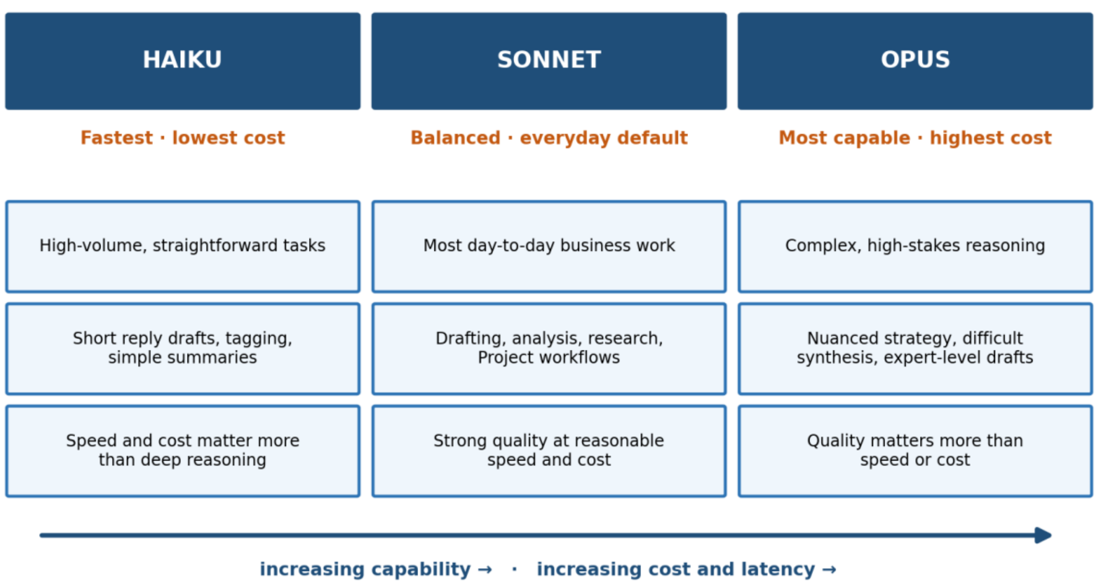


Figure 5 — The model family: match the tier to the task, not to your ambition.

Objective 3.3 — Align Model Selection with Cost, Speed, and Quality

The exam guide's own Sample 2 makes the pattern explicit: a high volume of short customer-reply drafts, where speed and cost matter more than deep reasoning, belongs on a faster, lower-cost model. Always using the most capable model "to maximize quality" wastes the cost and latency budget on work that doesn't need it; disabling features or switching platforms doesn’t address the trade-off at all. The reverse error also appears: routing genuinely complex strategy synthesis to the cheapest tier and then blaming the tool.

> **Analogy:** Vehicles, not trophies. You don’t take a freight truck to fetch coffee, and you don’t move a warehouse by bicycle. Nobody asks which vehicle is “best” — they ask what the trip needs. Model selection is the same question.

Objective 3.4 — Manage Context Limitations and Memory

What it means: the context window fills as conversations grow. Symptoms of the squeeze: Claude forgets constraints you set early, re-asks answered questions, or drifts from your specifics toward generic patterns. Three remedies, matched to situations:

Situation	Right move
Long chat has drifted; the task continues but early details are fading	Ask Claude to produce a structured summary of decisions, constraints, and key facts — then start a fresh chat seeded with that summary
The same background keeps being re-supplied across many chats	Persist it: move durable context into Project instructions and knowledge so every chat starts informed
A huge document plus a long discussion, and only one section still matters	Trim: start clean with just the relevant section rather than dragging the whole history forward

> **Exam tip:** “Restart and lose everything” and “push on in the degraded chat” are both distractors when a summary-then-restart or persist-to-Project option is available. The credited answer preserves the key facts while shedding the bulk.

7. Domain 4: Workflow Integration and Solution Design (16%)

The second-biggest domain, and the one closest to the credential's purpose statement: not "can you chat with Claude" but "can you improve how work gets done." It tests use-case judgment, process thinking, and the ability to be honest with stakeholders about both value and limits.

Objective 4.1 — Analyze Requirements and Identify Use Cases

What makes a good first use case? Look for four properties:

- **Text-heavy:** the work already lives in documents, emails, tickets, transcripts.
- **Repetitive:** the same shape of task recurs, so one good prompt pays off many times.
- **Verifiable:** a human can check the output quickly against something authoritative.
- **Draft-natured:** the output is a starting point a human finishes — not a final authority.

Weekly status synthesis, first-draft communications, meeting-notes-to-actions, survey theme extraction, document comparison: strong candidates. Final hiring decisions, unreviewed regulated filings, precision arithmetic at scale: weak or inappropriate ones (Domain 6 picks this thread up).

Use Claude to analyze the requirements themselves: describe the process and ask Claude to interview you — volumes, exceptions, quality bars, handoffs — before proposing a design. Surfacing the unstated requirement early is cheaper than discovering it in week three. (Architect-track readers will recognize this as the interview pattern; it starts here.)

Objective 4.2 — Use Claude for Research, Planning, and Process Optimization

Beyond executing tasks, Claude is a thinking partner for the work about the work: mapping a current process step by step and identifying friction, drafting project plans with dependencies and risks, synthesizing stakeholder input into requirements, and pressure-testing a plan ("what would make this fail?"). The tested behavior is supplying real material — the actual process description, the actual constraints — rather than asking for generic best practices, and then validating the plan against local reality before circulating it.

Objective 4.3 — Support Solution Design, Development, and Iteration

Pilot → measure → refine → scale. The credited pattern for introducing a Claude-based workflow is always incremental: design the solution around a narrow slice, run it alongside the existing process, measure quality and time saved, refine the prompts and instructions from real feedback, and only then expand. Big-bang rollouts — automate the whole function at once, skip the baseline, remove the human checkpoint on day one — are the standing distractor family in this domain.

Objective 4.4 — Integrate Claude into Existing Workflows: Augment or Redesign

Approach	What it means	When it fits
Augment	Keep the process shape; insert Claude at high-friction steps (first drafts, summarization, triage) with humans owning the checkpoints	The process is sound but slow; risk tolerance is low; you're building trust and a baseline
Redesign	Re-sequence the process around what AI makes cheap — e.g., synthesize-then-review replacing serial manual compilation	The old shape existed only because synthesis was expensive; augmentation has plateaued

Augment first is the usual right answer for a team new to AI — it produces evidence, surfaces failure modes safely, and earns the mandate for deeper redesign.

Objective 4.5 — Communicate Claude's Value and Limitations to Stakeholders

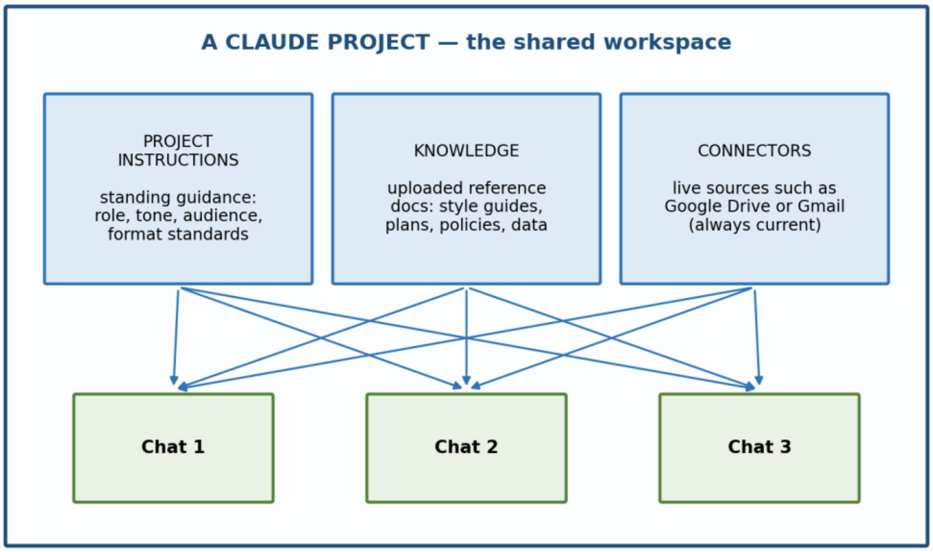
The honest pitch has three parts: what it does (concrete tasks, measured time savings), where it fails (hallucination risk, staleness, uneven quality on edge cases), and how the workflow controls for that (verification steps, human review gates, escalation paths). Overpromising is the fastest way to lose a stakeholder at the first visible error; a leader told upfront that outputs are drafts pending review treats the first error as the system working.

> **Exam tip:** Stakeholder-communication questions reward balanced, specific framing. Distractors either evangelize ("it eliminates the need for review") or catastrophize ("too risky to use for anything important"). The credited answer names the value AND the control.

8. Domain 5: Configuration and Knowledge Management (12%)

This domain is about setting Claude up so good behavior is the default — standing instructions written once, reference knowledge attached where the work happens, live connectors where freshness matters, and a maintenance habit so none of it rots.

Objective 5.1 — Configure Projects with Instructions and Knowledge



Every chat inside the Project inherits the instructions, knowledge, and connectors automatically — no re-pasting, no re-explaining.

Figure 6 — Anatomy of a Project: every chat inside inherits the instructions, knowledge, and connectors.

The division of labor to internalize: Project instructions carry what is durable — role, audience, tone, format standards, definitions, boundaries. Individual chat prompts carry what is task-specific — this document, this deadline, this deliverable. If you find yourself pasting the same paragraph into every chat, that paragraph is instructions material; if a chat's guidance would be wrong for the next chat, it is not.

A real Project instructions block is just plain text:

```
You support the Northwind account team.
- Audience: Northwind's operations leadership; formal but plain English
- Always use UK spelling and DD-MM-YYYY dates
- Status updates follow: Wins / Risks / Asks, max one page
- Financial figures come only from the attached Q-reports;
  if a figure isn't there, say so rather than estimating
- Never share pricing terms from other accounts
```

Notice the last two lines: instructions are also where you encode verification behavior ("say so rather than estimating") and boundaries — Domain 2 and Domain 6 thinking, installed as configuration.

Objective 5.2 — Manage Uploaded Knowledge and Connectors

Source type	Nature	Right for	Failure mode to manage
Uploaded knowledge	A snapshot — frozen at upload time	Stable references: style guides, policies, templates, past examples	Staleness: the source changed, the upload didn't
Connector (e.g., Google Drive, Gmail)	A live link — read at the source, current at time of use	Fast-changing content: working documents, plans in flux, mail threads	Scope: connect what the work needs, not everything you have access to

The classic scenario: a Project answers from last quarter's plan because the knowledge file is the March version while the team moved on in May. The credited fix is a maintenance habit — replace superseded files, remove obsolete ones, and prefer a connector for documents that change on a cadence. More knowledge is not better knowledge: irrelevant or duplicate files dilute attention and invite answers grounded in the wrong document.

Objective 5.3 — Write Effective System-Level Instructions

- **Be concrete, not aspirational:** "max one page, Wins/Risks/Asks" beats "be concise and well-organized." Vague instructions reproduce the vagueness problem they were meant to solve.
- **Positive instructions outperform bare prohibitions:** say what to do ("cite the source document for every figure") alongside what to avoid.
- **Include the boundary behavior:** what Claude should do when it doesn't know — say so, ask, or flag — is the highest-value line in most instruction sets.
- **Keep instructions and knowledge in their lanes:** guidance lives in instructions; reference content lives in knowledge. Pasting a 30-page policy into the instructions field burdens every chat with it.

> **Exam tip:** Instructions are probabilistic guidance, not enforcement — strong instructions raise consistency, they don't guarantee it. High-stakes rules still need verification and human checkpoints (Domain 2). Architect-track readers will meet the deterministic version of this idea as hooks and gates; at the Associate level, the control is the human.

Objective 5.4 — Maintain and Update Configurations Over Time

Configuration is a garden, not a monument. The tested habits: review instructions when outputs drift or the team's standards change; version important instruction sets so you can see what changed when quality changed; prune knowledge on a schedule; and treat recurring correction patterns in chats ("we keep fixing the same thing") as a signal that the fix belongs upstream in the configuration, not downstream in every conversation.

9. Domain 6: Governance, Risk, and Responsible Use (15%)

The conscience of the exam, and — at 15% — a heavyweight. The tested mindset is neither recklessness nor refusal: it is finding the compliant path that keeps the work moving. Almost every question in this domain has the same skeleton: someone wants to do something useful with data or AI output; one option bypasses policy, one abandons the task, and one enables it safely. Pick the enabler.

Objective 6.1 — Identify Appropriate and Inappropriate Use Cases

Appropriate (with normal diligence)	Inappropriate or requiring special controls
Drafting, summarizing, and adapting business content a human reviews	Presenting AI output as final authority in regulated filings without qualified review
Research synthesis, brainstorming, process mapping, planning support	Sole-source decisions about people: hiring, firing, performance, credit, medical
Analyzing anonymized or non-sensitive data for trends	Feeding regulated personal data into tools outside approved policy paths
Meeting notes → actions, tickets → themes, documents → comparisons	High-precision computation at scale where a spreadsheet/database is the right tool

The pattern behind the right-hand column: consequence without accountability. Where output affects people's rights, money, health, or legal standing, a qualified human must own the decision — Claude assists, humans decide.

Objective 6.2 — Apply Data Sensitivity, Regulatory, and Privacy Considerations

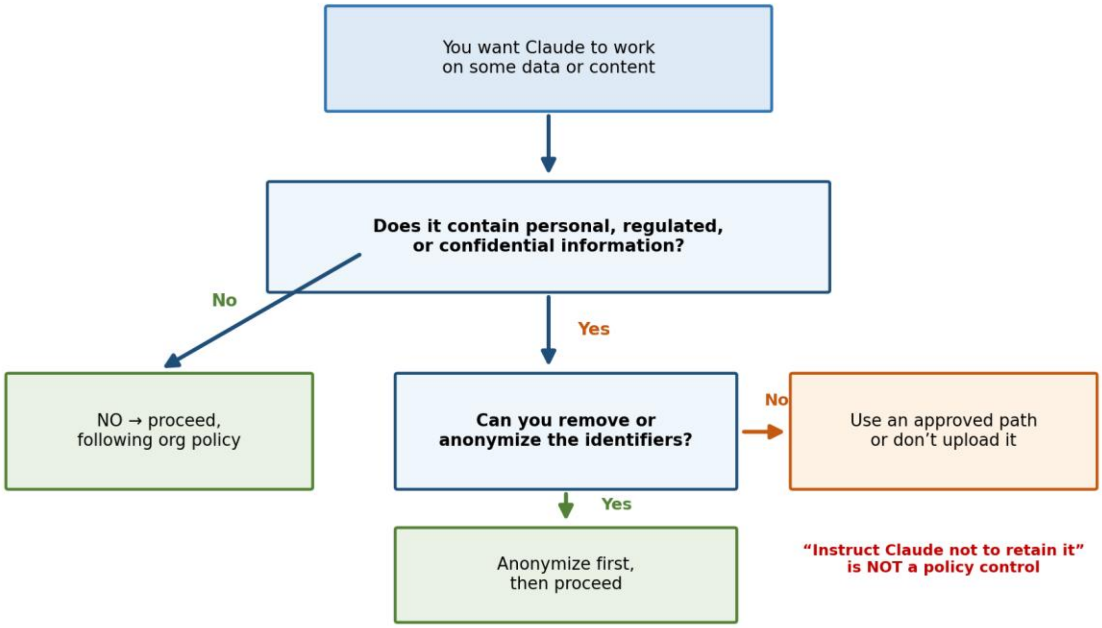


Figure 7 — The pre-flight check that should run before any data reaches any AI tool.

The exam guide's own Sample 3 is the template: a project manager wants to upload a spreadsheet of customer names and account numbers for trend analysis, but policy restricts sharing regulated personal data. The credited action is to remove or anonymize the identifiers first — the trends survive anonymization; the policy exposure doesn't. Uploading as-is violates policy. "Instruct Claude not to retain it" is not a policy control — a chat instruction cannot substitute for a governance safeguard. And skipping the analysis entirely surrenders value the compliant path could have delivered.

> **Analogy:** A hospital researcher studying treatment outcomes doesn't take patient charts home — and doesn't abandon the study either. They work with de-identified records: the science proceeds, the privacy obligation holds. Anonymization is how you say yes safely.

Objective 6.3 — Follow Organizational AI Policies and Governance

- **Know your organization's rules before the moment of need:** which tools and workspaces are approved, which data classes may be used where, what requires disclosure or sign-off.
- **Policy beats convenience, every time it's tested:** "the analysis is internal" and "everyone does it" are distractor rationales, not controls.
- **When policy is silent, escalate — don't improvise:** a novel use case that policy doesn't address goes to whoever owns AI governance, before the data moves. (The Architect exam calls the same instinct "the agent must not invent policy." Neither should you.)
- **Use the approved path when one exists:** enterprise workspaces and sanctioned connectors exist precisely so sensitive work has a home.

Objective 6.4 — Understand the Ethical Implications of AI Use

Beyond written policy sit four durable obligations. Accountability: you own what you send — “the AI wrote it” transfers nothing. Transparency: disclose AI assistance where your organization, audience, or context expects it. Fairness: outputs can inherit bias from training data or skewed inputs; work affecting people gets screened for it (Domain 2’s bias lens, applied with stakes). Proportional human oversight: the greater the potential harm of an error, the earlier and more senior the human in the loop.

> **Exam tip:** Governance answers are recognizable by their shape: they name a concrete safeguard (anonymize, use the approved workspace, obtain sign-off, disclose) and the task still gets done. If an option’s only virtue is speed, or its only content is refusal, it’s a distractor.

10. Domain 7: Troubleshooting and Optimization (10%)

The smallest domain, but it synthesizes everything before it: when output disappoints, diagnose the actual cause and apply the proportionate fix — usually in the prompt, the context, or the configuration, and only rarely in the model choice.

Objective 7.1 — Diagnose Underperforming Prompts and Poor Outputs

The diagnostic table — symptom to root cause to fix:

Symptom	Likely root cause	Proportionate fix
Generic, could-be-anyone output	Missing context, audience, or criteria in the prompt	Add the missing building blocks (Figure 3); supply source material
Wrong structure or format	No format specification or example	Specify the format; better, show one example of the desired shape
Confidently wrong specifics	Hallucination — asked for facts without grounding	Ground in authoritative source documents; verify load-bearing claims (Domain 2)
Forgets earlier constraints; re-asks answered questions	Context-window squeeze in a long chat	Summarize-and-restart, or persist durable context to a Project (Domain 3)
Quality varies wildly across similar requests	Underspecified prompt — each run guesses differently	Add explicit criteria and a worked example; templatize the prompt
Answers grounded in outdated information	Stale Project knowledge or missing live source	Update/replace knowledge files; use a connector for fast-changing content (Domain 5)
Uneven quality across parts of one big request	Mega-prompt doing too many jobs at once	Decompose into sequenced steps with checkpoints (Domain 1)

Exam tip: Diagnosis order matters and is itself tested: prompt → context → configuration → model. “Switch to a bigger model” is the credited answer only when the task genuinely exceeds the current tier’s reasoning — it is never the fix for vagueness, staleness, or missing grounding.

Objective 7.2 — Adjust Your Approach Based on Feedback

Treat results as data. When an output misses, identify specifically what missed — facts, tone, structure, completeness — and feed that back precisely (Domain 1’s iteration discipline). When the same correction recurs across chats, stop patching conversations and fix the system: move the rule into Project instructions, add the missing reference to knowledge, or restructure the prompt template. One-off errors get one-off feedback; patterns get configuration changes.

Objective 7.3 — Optimize Workflows for Efficiency

- **Templatize what recurs:** any prompt you've written three times becomes a saved template with blanks for the variables.
- **Consolidate context once:** recurring background belongs in a Project, not in every prompt — less typing, more consistency.
- **Right-size the model per step:** in a multi-step workflow, route the high-volume simple steps to a fast, low-cost tier and reserve the capable tier for the hard synthesis step.
- **Measure before and after:** time saved, revision rounds, error rates — optimization claims need baselines, and stakeholders (Domain 4) will ask.
- **Know the escalation boundary:** when the optimization the workflow really needs is an API integration, automation without a human at the keyboard, or an agent — that's the handoff to Architects and Developers, and recognizing it is the Associate's final scored skill.

11. Sample Question Walkthroughs: Learning the Exam's Logic

The exam guide publishes three sample items. Rather than repeat them verbatim, here is the reasoning pattern behind each — the transferable logic that answers questions you've never seen.

Sample (paraphrased)	Correct logic	Why the distractors fail
S1 · Domain 2: Claude confidently summarizes a regulation, citing a specific subsection; the summary is bound for the compliance team	Verify the cited subsection against the official regulation text before sharing — high stakes demand grounding in the authoritative source	"Send as-is, it sounded confident" and "ask Claude to rate its confidence" both treat confidence as evidence — it isn't; rewording to sound formal polishes the packaging without touching correctness
S2 · Domain 3: high volume of short customer-reply drafts; speed and cost matter more than deep reasoning	Use a faster, lower-cost model suited to straightforward, high-volume work	The most capable model on every reply burns cost and latency for no quality gain; disabling features doesn't address the trade-off; switching platforms is a non-answer
S3 · Domain 6: uploading a spreadsheet of customer names and account numbers for trend analysis, against a policy restricting regulated personal data	Remove or anonymize the identifiers first, consistent with policy — the analysis proceeds, the exposure doesn't	Uploading as-is violates policy; "instruct Claude not to retain it" is a chat instruction, not a governance control; skipping the analysis abandons value the compliant path delivers

Pattern recognition — the five reflexes that answer unseen questions:

- **Confidence is never evidence.** Any option that trusts, rates, or reformats confidence instead of verifying against a source is wrong.
- **Proportionate beats maximal.** Fix the prompt before the model, augment before redesigning, pilot before scaling. The biggest hammer is bait.
- **Enable within policy.** The governance winner names a safeguard and still gets the task done; bypass and abandonment both lose.
- **Fix upstream.** Recurring problems get configuration fixes (instructions, knowledge, templates), not per-chat patching.
- **Humans own consequences.** Where output affects people, money, or legal standing, the right answer includes a qualified human before it takes effect.

Exam tip: Mechanics matter too: the exam mixes multiple-choice with multiple-response items, and each multiple-response item states how many responses to select — read that line before the options, and select exactly that many. With 60 items in 120 minutes you have two minutes each; bank time on the easy ones.

12. Putting It to Work: Daily Claude Habits

Everything above doubles as a practical playbook. These habits improve your day-to-day results immediately — each mapped to the domain it reinforces, so practice and exam prep are the same activity.

Habit	What to do	Domain
Brief like a manager	Every prompt carries the five blocks: role/context, task, source material, format, constraints/audience	D1
Decompose the big stuff	Multi-part work becomes sequenced steps with a checkpoint after each	D1
Verify the load-bearing facts	Before anything ships, check the numbers, names, dates, and citations the decision rests on — against the authoritative source	D2
Escalate by stakes, not vibes	Compliance, legal, finance, external, people-decisions → qualified human review, always	D2
Right-size the model	Haiku-class for volume, Sonnet-class for the everyday, Opus-class for the genuinely hard	D3
Summarize-and-restart long chats	When a chat drifts, extract a structured summary of decisions and facts, then begin fresh with it	D3
Pilot, measure, then scale	New workflow ideas start on a narrow slice with a baseline and a human checkpoint	D4
Tell stakeholders both halves	Present the value AND the failure modes AND the control that manages them	D4
Move repeated context upstream	Anything you've pasted into three chats belongs in Project instructions or knowledge	D5
Prune the knowledge garden	Replace superseded files; use connectors for fast-changing sources	D5
Run the data pre-flight	Personal, regulated, or confidential? Anonymize or use the approved path before anything is uploaded	D6
Diagnose before you switch	Weak output → check prompt, then context, then configuration — the model comes last	D7

13. One-Page Cheat Sheet: Facts to Memorize

Topic	The fact
Exam format	60 items · multiple-choice AND multiple-response (each states how many to select) · 120 minutes · proctored (online or Pearson VUE center)
Scoring	Criterion-referenced · scaled 100–1,000 · pass = 720 · domain percentages on the score report are diagnostic only
Logistics	$99 per attempt · reschedule/cancel ≥24h before or forfeit the fee · government photo ID matching registration name exactly · accommodations approved BEFORE scheduling
Retakes	Wait 14 days after 1st fail, 30 after 2nd, 90 after 3rd · max 4 attempts per rolling 12 months
Validity & renewal	Credential valid 12 months · on-time renewal = free non-proctored assessment · lapsed = full exam at full fee
Domain weights	D2 Evaluation 21% · D4 Workflow 16% · D6 Governance 15% · D1 Prompting 14% · D3 Selection 12% · D5 Configuration 12% · D7 Troubleshooting 10%
Prompting	Five blocks: role/context · task · source material · format · constraints/audience — every omission is a guess
Decomposition	Big requests → sequenced steps with checkpoints; fix uneven mega-prompt output with structure, not a bigger model
Hallucination	Confident fabricated specifics; tell = misplaced precision; cure = verify against the authoritative source — NEVER self-reported confidence
Human review	Mandatory for compliance, legal, finance, external publication, and decisions about people
Output format	Inline = conversational · artifact = reusable deliverable · structured data = feeds another tool
Features	One-off → chat · recurring context → Project · deep multi-source → research mode · iterated deliverable → artifact · recurring specialized task → skill · data analysis / real files → code execution
Models	Haiku = fast/cheap/high-volume · Sonnet = balanced default · Opus = most capable, complex reasoning — match tier to task
Context squeeze	Symptoms: forgotten constraints, generic drift · remedies: summarize-and-restart, persist to Project, trim to what matters
Workflow design	Good use cases: text-heavy, repetitive, verifiable, draft-natured · augment before redesign · pilot → measure → scale
Configuration	Durable guidance → Project instructions · reference content → knowledge · fast-changing sources → connectors · prune for staleness
Governance	Anonymize/redact regulated identifiers before upload · “tell Claude not to retain it” is NOT a control · policy silent → escalate, don’t improvise

Topic	The fact
Ethics	You own what you send · disclose per policy · screen people-affecting output for bias · oversight scales with stakes
Troubleshooting order	Prompt → context → configuration → model (last) · recurring corrections = fix the configuration, not the chat
Escalation boundary	APIs, agents, automation, custom integrations → hand off to Claude Architects / Developers

14. A Two-Week Study Plan

Days	Focus	Activity
1–2	Foundations	Read Chapters 1–3 of this tutorial (newcomers: extra time on Section 3.1); read the official exam guide end-to-end; make flashcards from the glossary and the format/scoring table
3–4	Domains 1 & 2 (35% of exam)	Study Chapters 4–5; in Claude, rewrite three of your recent weak prompts with the five blocks; take one real output and run the full verify-before-you-ship loop on it
5–6	Domains 3 & 5	Study Chapters 6 & 8; build a real Project — instructions, two knowledge files, one connector if available — and run three chats inside it; practice a summarize-and-restart on a long chat
7–8	Domain 4 hands-on	Study Chapter 7; pick one recurring task in your own work, design an augment-style workflow with a human checkpoint, and pilot it; draft the two-sided stakeholder pitch
9–10	Domains 6 & 7	Study Chapters 9–10; walk the data pre-flight (Figure 7) against three datasets you actually handle; work the symptom→cause→fix table from memory
11–12	Sample questions & exam logic	Work Chapter 11; for each official sample item, articulate WHY each distractor fails before reading the rationale; drill the five reflexes
13	Full review	Run the blueprint self-assessment checklist (Chapter 15) and re-study every objective you can't tick; re-read the cheat sheet (Chapter 13) and the three recurring themes (Chapter 1)
14	Rest & logistics	Confirm ID, workspace, and Pearson VUE check-in requirements; light review only; sleep

Good luck — and remember the exam's underlying question is always the same: ***what is the proportionate, verifiable, policy-compliant action that keeps the work moving?***

When you pass: the natural next step in this series is the Claude Certified Architect – Foundations tutorial — where the judgment you built here (verify, decompose, configure, govern) gets rebuilt as engineering: agentic loops, tools, hooks, and multi-agent systems. You'll recognize almost everything.

15. Appendix: Blueprint Coverage and Self-Assessment Checklist

The exam guide's preparation advice is explicit: study the blueprint and *self-assess against each objective*. This appendix is that instrument. All 30 blueprint objectives are listed below with the tutorial section that covers each. Work down the "Can you…?" column honestly: tick an objective only if you could act on it right now, without notes, in a real scenario. Anything unticked tells you exactly which section to re-read — and, better, which skill to practice hands-on in Claude before exam day.

Domain	Blueprint objective — can you…?	Covered in	✓
D1 · 14%	Write effective prompts for business and technical tasks	Ch 4, Obj 1.1	
D1	Decompose a complex request into structured, sequenced steps	Ch 4, Obj 1.2	
D1	Iterate a prompt to improve output quality	Ch 4, Obj 1.3	
D1	Adapt prompting strategy to the task type — analysis, research, drafting, brainstorming	Ch 4, Obj 1.4	
D2 · 21%	Evaluate an output for accuracy and completeness	Ch 5, Obj 2.1	
D2	Spot hallucinations, internal inconsistencies, and bias	Ch 5, Obj 2.2	
D2	Apply fact-checking and validation techniques	Ch 5, Obj 2.3	
D2	Judge when human review or extra verification is required	Ch 5, Obj 2.4	
D2	Edit, adapt, refine, and compare outputs for the intended audience	Ch 5, Obj 2.5	
D2	Organize and curate information; choose artifact vs. inline vs. structured data	Ch 5, Obj 2.6	
D3 · 12%	Select the right product feature — Projects, research mode, chat, artifacts	Ch 6, Obj 3.1	
D3	Differentiate the model types — Haiku, Sonnet, Opus	Ch 6, Obj 3.2	
D3	Align model selection with cost, speed, and quality requirements	Ch 6, Obj 3.3	
D3	Manage context limits and memory — when to restart, summarize, or persist	Ch 6, Obj 3.4	
D4 · 16%	Analyze requirements and identify good use cases	Ch 7, Obj 4.1	
D4	Use Claude for research, planning, and process optimization	Ch 7, Obj 4.2	
D4	Support solution design, development, and iteration	Ch 7, Obj 4.3	

Domain	Blueprint objective — can you…?	Covered in	✓
D4	Integrate Claude into existing workflows — augment or redesign	Ch 7, Obj 4.4	
D4	Communicate Claude's value and limitations to stakeholders	Ch 7, Obj 4.5	
D5 · 12%	Configure a Project with instructions and knowledge sources	Ch 8, Obj 5.1	
D5	Manage uploaded knowledge and connectors (e.g., Google Drive, Gmail)	Ch 8, Obj 5.2	
D5	Write effective system-level instructions	Ch 8, Obj 5.3	
D5	Maintain and update configurations, knowledge, and instructions over time	Ch 8, Obj 5.4	
D6 · 15%	Identify appropriate and inappropriate use cases	Ch 9, Obj 6.1	
D6	Apply data-sensitivity, regulatory, and privacy considerations	Ch 9, Obj 6.2	
D6	Follow organizational AI policies and governance standards	Ch 9, Obj 6.3	
D6	Reason about the ethical implications of AI use	Ch 9, Obj 6.4	
D7 · 10%	Diagnose and resolve underperforming prompts and poor outputs	Ch 10, Obj 7.1	
D7	Adjust your approach based on feedback and results	Ch 10, Obj 7.2	
D7	Optimize workflows for efficiency and effectiveness	Ch 10, Obj 7.3	

A note on where questions really come from: items are written against these 30 objectives and nothing else. If a practice question seems to test something outside this list — API parameters, agent architectures, coding — it belongs to the Architect or Developer exams, not this one. Conversely, if you can honestly tick all 30 rows AND have the hands-on experience the guide recommends, you match the minimally qualified candidate profile the passing standard was set against.

> **Exam tip:** Pair this checklist with your score-report preview: the exam reports percent-correct per domain. Weight your remaining study time by (domain weight) × (your weakness) — a shaky Domain 2 at 21% costs far more scaled-score points than a shaky Domain 7 at 10%.